

B.TECH · 1ST YEAR · 1ST SEMESTER

Linked Lists

Unit II – Data Structures

 Complete Revision Sheet & Exam Guide

Operations • Algorithms • Complexity • Flashcards • Exam Tips

 Singly LL

 Doubly LL

 Circular LL

 Complexity

 Flashcards

Table of Contents

1. Introduction to Linked Lists
2. Node Representation & Structure
3. Singly Linked List – All Operations
4. Doubly Linked List
5. Circular Linked List
6. Comparison Tables
7. Applications & Complexity Summary
8. Key Terms (40 Definitions)
9. Syntax & Code Reference Box
10. Problem-Solving Patterns
11. Flashcards (35 Q&A)
12. Exam Tips & Quick Revision

 DEFINITION

A **Linked List** is a linear data structure consisting of a sequence of **nodes**, where each node stores a **data element** and a **pointer (link)** to the next node in memory. Nodes are stored at *non-contiguous* memory locations.

 Why Linked Lists over Arrays?

Problem with Arrays	Solution in Linked Lists
Fixed size – must declare in advance	Dynamic size – grows/shrinks at runtime
Insertion/Deletion costly – shifting required	Efficient – just update pointers
Memory wasted if array is partially used	Only allocates memory when needed
Contiguous memory block needed	Scattered memory locations work fine

 Real-World Analogy TRAIN COACHES ANALOGY

Think of a linked list as a **train**. Each **coach (node)** has passengers (data) and a coupling hook (pointer) connecting it to the next coach. The engine is the **head**. You cannot jump to coach 5 directly – you must pass through $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$.

 Types of Linked Lists

1. SINGLY LINKED LIST

Each node has one pointer: **next**. Traversal is **one-directional** only.

```
[Data|Next] → [Data|Next] →
                [Data|NULL]
```

3. CIRCULAR LINKED LIST

Last node's **next** points back to **first node**. No NULL pointer.

```
[Data|Next] → [Data|Next]
↑               ↓
```

2. DOUBLY LINKED LIST

Each node has two pointers: **prev** and **next**. Traversal is **bi-directional**.

```
NULL ← [P|Data|N] ⇔ [P|Data|N] → NULL
```

 KEY PROPERTIES

- Non-contiguous memory allocation
- Dynamic size (runtime growth)
- Pointer-based traversal (no index)
- Extra memory for pointer storage

Node Structure (C Syntax)

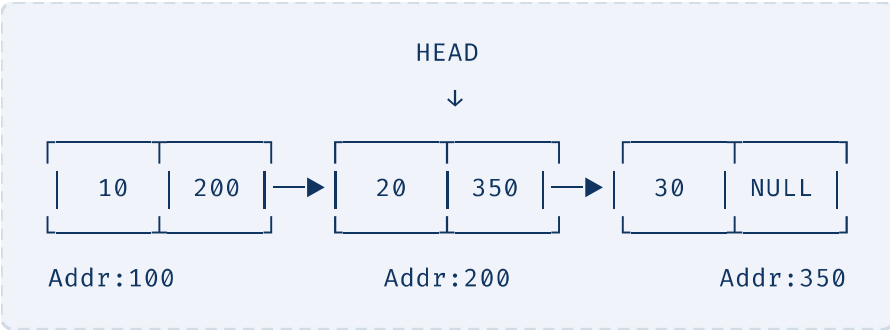
```
struct Node {
    int data;           // stores the value
    struct Node* next; // pointer to next node
};

// For Doubly Linked List:
struct DNode {
    int data;
    struct DNode* prev;
    struct DNode* next;
};
```

Creating a New Node

```
struct Node* newNode =
    (struct Node*) malloc(sizeof(struct Node));
newNode->data = 10;
newNode->next = NULL;
```

Memory Visualization



HEAD POINTER

- **Head** = pointer to *first node*
- If `head == NULL` → list is empty
- Never lose the `head` pointer!
- All traversals start from `head`

Terminologies

Term	Meaning
Node	Basic unit; contains data + pointer
Head	Pointer to first node; entry point of list
Tail	Last node; its next = NULL (or head in circular)
NULL	Indicates end of list; no further node
Traversal	Visiting each node from head to tail
Predecessor	Node just before the current node
Successor	Node just after the current node

+ 3.1 Insert at Beginning $O(1)$

ALGORITHM STEPS

- 1 Create a new node with given data
- 2 Set `newNode→next = head`
- 3 Update `head = newNode`

```
void insertBegin(int data) {
    Node* newNode = malloc(sizeof(Node));
    newNode→data = data;
    newNode→next = head; // point to old head
    head = newNode;      // update head
}
```

Before: HEAD → [10] → [20] → NULL

Insert 5:

[5|?] → create new node
 [5|→] → [10] → [20] → NULL
 HEAD ↑ (move here)

After: HEAD → [5] → [10] → [20] → NULL

+ 3.2 Insert at End $O(n)$

ALGORITHM STEPS

- 1 Create a new node
- 2 If list empty: `head = newNode`
- 3 Traverse to last node (where `next=NULL`)
- 4 Set `last→next = newNode`

```
void insertEnd(int data) {
    Node* newNode = malloc(sizeof(Node));
    newNode→data = data;
    newNode→next = NULL;
    if (head == NULL) { head = newNode; return; }
    Node* temp = head;
    while (temp→next != NULL)
        temp = temp→next; // reach last node
    temp→next = newNode;
}
```

HEAD → [10] → [20] → [30] → NULL

Traverse until next == NULL
 temp points to [30]

[30].next = [40] (new node)

After: HEAD → [10] → [20] → [30] → [40] → NULL

+ 3.3 Insert at Position $O(n)$

ALGORITHM STEPS

- 1 Create new node
- 2 Traverse to node at position `pos - 1`
- 3 Set `newNode→next = prev→next`

Insert 25 at position 3:

HEAD → [10] → [20] → [30] → NULL

Traverse to pos 2 → temp = [20]

4 Set `prev->next = newNode`

```
void insertPos(int data, int pos) {  
    Node* newNode = malloc(sizeof(Node));  
    newNode->data = data;  
    Node* temp = head;  
    for(int i=1; i<pos-1; i++)  
        temp = temp->next;  
    newNode->next = temp->next;  
    temp->next = newNode;  
}
```

`newNode.next = [20].next = [30]`

`[20].next = [25]`

After: `[10] → [20] → [25] → [30] → NULL`

✗ 3.4 Delete from Beginning $O(1)$

ALGORITHM STEPS

- 1 Check if list is empty
- 2 Store `temp = head`
- 3 Move `head = head→next`
- 4 Free memory: `free(temp)`

```
void deleteBegin() {
    if (head == NULL) return; // empty list
    Node* temp = head;
    head = head→next; // move head forward
    free(temp); // release memory
}
```

Before: HEAD → [10] → [20] → [30] → NULL

```
temp = [10]
head = head.next → [20]
free(temp) → delete [10]
```

After: HEAD → [20] → [30] → NULL

✗ 3.5 Delete from End $O(n)$

ALGORITHM STEPS

- 1 If empty or single node: handle separately
- 2 Traverse with two pointers: `prev` & `curr`
- 3 Stop when `curr→next == NULL`
- 4 Set `prev→next = NULL`
- 5 Free `curr`

```
void deleteEnd() {
    if (head == NULL) return;
    if (head→next == NULL) { free(head); head=NULL; return; }
    Node* prev = NULL, *curr = head;
    while (curr→next != NULL) {
        prev = curr;
        curr = curr→next;
    }
    prev→next = NULL;
    free(curr);
}
```

HEAD → [10] → [20] → [30] → NULL

```
Traverse: prev=[20], curr=[30]
curr.next == NULL → stop
prev.next = NULL
free(curr) → delete [30]
```

After: HEAD → [10] → [20] → NULL

✗ 3.6 Delete Specific Node $O(n)$

```
void deleteNode(int key) {
    Node* prev = NULL, *curr = head;
    while (curr != NULL && curr→data != key) {
```

```

    prev = curr; curr = curr->next;
}
if (curr == NULL) return; // not found
if (prev == NULL) head = curr->next; // first node
else prev->next = curr->next;
free(curr);
}

```

3.7 Traversal O(n)

```

void traverse() {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

```

3.8 Searching O(n)

```

int search(int key) {
    Node* temp = head;
    int pos = 1;
    while (temp != NULL) {
        if(temp->data == key) return pos;
        temp = temp->next; pos++;
    }
    return -1; // not found
}

```

Complexity Table – Singly Linked List

Operation	Time Complexity	Space	Reason
Insert at Beginning	O(1)	O(1)	Only head pointer changes
Insert at End	O(n)	O(1)	Must traverse to last node
Insert at Position	O(n)	O(1)	Traverse to position first
Delete from Beginning	O(1)	O(1)	Only head pointer changes
Delete from End	O(n)	O(1)	Must traverse to find last-1
Delete Specific	O(n)	O(1)	Linear search required
Traversal	O(n)	O(1)	Visit all n nodes
Search	O(n)	O(1)	Linear search

 DEFINITION

A DLL is a linked list where each node has **two pointers**: **prev** (to previous node) and **next** (to next node). Allows **bidirectional traversal**.

 Structure

```
struct DNode {
    int data;
    struct DNode* prev;
    struct DNode* next;
};
```

 DLL Diagram
$$\text{NULL} \leftarrow [P|10|N] \rightleftharpoons [P|20|N] \rightleftharpoons [P|30|N] \rightarrow \text{NULL}$$

HEAD

↑

↑ TAIL

 Insert at Beginning $O(1)$

- 1 Create new node
- 2 Set `newNode→next = head`
- 3 Set `newNode→prev = NULL`
- 4 If head≠NULL: `head→prev = newNode`
- 5 Update `head = newNode`

```
void dllInsertBegin(int data) {
    DNode* n = malloc(sizeof(DNode));
    n→data = data;
    n→prev = NULL;
    n→next = head;
    if (head ≠ NULL) head→prev = n;
    head = n;
}
```

 Delete a Node $O(n)$

- 1 Find node to delete (traverse)
- 2 Update predecessor's next: `del→prev→next = del→next`
- 3 Update successor's prev: `del→next→prev = del→prev`
- 4 Handle edge cases (first/last node)
- 5 Free the deleted node

```
void dllDelete(DNode* del) {
    if (del→prev ≠ NULL)
        del→prev→next = del→next;
    else
        head = del→next; // deleting head
    if (del→next ≠ NULL)
        del→next→prev = del→prev;
    free(del);
}
```

 ADVANTAGES OF DLL

- Bidirectional traversal (forward & backward)
- Deletion easier – no need to track **prev** separately
- Efficient insertion before a given node
- Useful for implementing Deque

 DISADVANTAGES OF DLL

- Extra memory – 2 pointers per node
- More complex pointer manipulation
- Insertion & deletion involves updating 2 links
- Slightly slower due to extra overhead

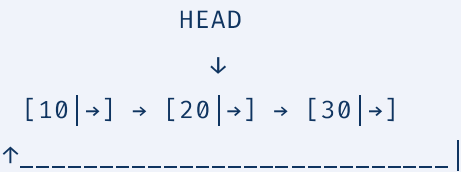
 DLL Complexity Table

Operation	Time	Space
Insert at Beginning	O(1)	O(1)
Insert at End	O(n)	O(1)
Delete (given node ptr)	O(1)	O(1)
Delete by value	O(n)	O(1)
Traversal (forward/backward)	O(n)	O(1)
Search	O(n)	O(1)

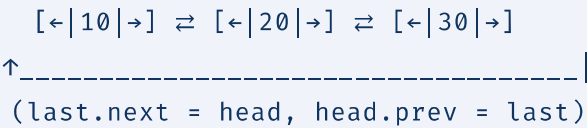
📖 DEFINITION

A **Circular Linked List** is a linked list where the **last node's next pointer** points back to the **first (head) node**, forming a circle. There is **no NULL** pointer in the list.

📊 Singly Circular LL



📊 Doubly Circular LL



⚙️ Insert at End (CLL)

```
void cllInsert(int data) {
    Node* n = malloc(sizeof(Node));
    n->data = data;
    if (head == NULL) {
        head = n;
        n->next = head; // points to itself
        return;
    }
    Node* temp = head;
    while (temp->next != head)
        temp = temp->next;
    temp->next = n;
    n->next = head; // circular link
}
```

🔄 Traversal (CLL)

```
void cllTraverse() {
    Node* temp = head;
    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head); // stop at head again
}
```

✅ ADVANTAGES OF CLL

- No NULL pointers – entire list can be traversed from any node
- Efficient for circular/looping operations
- Useful when list needs to be traversed repeatedly
- End-to-beginning connection without reset

🌐 APPLICATIONS OF CLL

- 🖥️ **Round-Robin CPU Scheduling** – processes in a cycle
- 🎵 **Music Player** – repeat all songs in loop
- 🎮 **Multiplayer Games** – player turns cycle
- 💾 **Buffer Management** – circular buffer/ring buffer

⚠️ Key Points – Circular List

Aspect	Singly Circular	Doubly Circular
Pointers per node	1 (next only)	2 (prev + next)
Last node points to	head (first node)	head (first node)

Traversal direction	One-way loop	Two-way loop
NULL pointers	None	None
Loop termination	<code>temp == head</code>	<code>temp == head</code>

Arrays vs Linked Lists

Feature	Arrays	Linked Lists
Memory Allocation	Contiguous block required	Non-contiguous (scattered)
Size	Fixed at declaration	Dynamic (grows/shrinks)
Memory Waste	May waste if under-used	Only uses what's needed
Access Time	$O(1)$ – direct index	$O(n)$ – sequential
Insertion (begin)	$O(n)$ – shift elements	$O(1)$ – update pointer
Insertion (end)	$O(1)$ – if space exists	$O(n)$ – traverse first
Deletion	$O(n)$ – shifting needed	$O(n)$ but no shifting
Cache Performance	Better (contiguous)	Poorer (scattered)
Extra Memory	None (no pointers)	Extra for pointer(s)
Binary Search	Possible – $O(\log n)$	Not applicable
2D structure	Easily possible	Complex

Types of Linked Lists Comparison

Feature	Singly LL	Doubly LL	Circular LL
Pointers per node	1 (next)	2 (prev, next)	1 (next, circular)
Memory per node	Less	More	Moderate
Traversal direction	Forward only	Forward & Backward	Forward (loop)
Last node points to	NULL	NULL	Head (first node)
Reverse traversal	Not possible	Possible	Not directly
Insert at beginning	$O(1)$	$O(1)$	$O(1)$
Delete (given ptr)	$O(n)$ – need prev	$O(1)$	$O(n)$
Applications	Stack, Queue, basic	Browser history, Deque	Round-robin, buffers
Complexity	Simple	Moderate	Moderate

Complexity Comparison – All DS

Operation	Array	Singly LL	Doubly LL
Access by index	$O(1)$	$O(n)$	$O(n)$

Search	$O(n)$	$O(n)$	$O(n)$
Insert at begin	$O(n)$	$O(1)$	$O(1)$
Insert at end	$O(1)^*$	$O(n)$	$O(n)$
Insert at position	$O(n)$	$O(n)$	$O(n)$
Delete from begin	$O(n)$	$O(1)$	$O(1)$
Delete from end	$O(1)$	$O(n)$	$O(n)$

* $O(1)$ amortized with dynamic arrays



Applications of Linked Lists

DATA STRUCTURE IMPLEMENTATION

- **Stack** – Insert/delete at head ($O(1)$)
- **Queue** – Insert at tail, delete at head
- **Deque** – Doubly LL for both-end ops
- **Hash Table** – Chaining uses linked list

REAL-WORLD USES

- **Browser History** – DLL for back/forward
- **Undo/Redo** – DLL in editors
- **Music Playlist** – Circular LL for repeat
- **OS Process List** – Round-robin scheduling

POLYNOMIAL REPRESENTATION

Each node: [coeff | exp | next]

e.g., $3x^2 + 5x + 2 \rightarrow [3|2] \rightarrow [5|1] \rightarrow [2|0] \rightarrow \text{NULL}$

MEMORY MANAGEMENT

- Free list maintenance by OS
- Dynamic memory allocation tracking
- Garbage collection linked lists
- File allocation tables

GRAPH – ADJACENCY LIST

Each vertex has a linked list of its neighbors. For vertex A connected to B, C, D:

$A \rightarrow [B] \rightarrow [C] \rightarrow [D] \rightarrow \text{NULL}$

Space efficient for sparse graphs: $O(V + E)$ vs $O(V^2)$ for matrix.



Complete Complexity Summary

Operation	Best Case	Average Case	Worst Case	Space
Access (by position)	$O(1)^*$	$O(n)$	$O(n)$	$O(1)$
Search (linear)	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Insert (beginning)	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Insert (end)	$O(1)^{**}$	$O(n)$	$O(n)$	$O(1)$
Insert (position)	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Delete (beginning)	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Delete (end/value)	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Traversal	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Reverse	$O(n)$	$O(n)$	$O(n)$	$O(1)$

* $O(1)$ only for head node | ** $O(1)$ if tail pointer is maintained

Overall space for a linked list of n nodes: $O(n)$ | Each node uses: `sizeof(data) + sizeof(pointer(s))` bytes
On 64-bit systems: pointer = 8 bytes | int = 4 bytes | So each SLL node = 12 bytes minimum.

Linked List → Linear DS of nodes connected via pointers

Head → Pointer to first node; entry point of list

Pointer → Variable that stores memory address

Traversal → Visiting every node from head to tail

Doubly LL → Each node has prev + next pointers

Dynamic Memory → Memory allocated at runtime (heap)

free() → C function to release allocated memory

Predecessor → Node immediately before given node

Insertion → Adding a new node into the list

Search → Finding a node with given key value

Contiguous → Consecutive memory addresses (arrays)

Sequential Access → Must visit nodes one by one

Time Complexity → How execution time grows with input

O(1) → Constant time; independent of input size

Stack → LIFO structure; implementable via LL

Deque → Double-ended queue; uses doubly LL

Round-Robin → CPU scheduling using circular LL

Slow Pointer → Moves 1 step at a time (Floyd's)

Cycle/Loop → Node's next points to previous node

Middle Node → Node at position $\lfloor n/2 \rfloor$ in list

Node → Basic unit: stores data + pointer(s)

Tail → Last node; its next is NULL (or head)

NULL → Special value indicating no node / end of list

Singly LL → Each node has one pointer (next only)

Circular LL → Last node points back to first node

malloc() → C function to allocate memory dynamically

struct → User-defined data type in C for node

Successor → Node immediately after given node

Deletion → Removing a node from the list

Bidirectional → Can traverse both forward and backward

Non-contiguous → Nodes stored at random addresses

Random Access → Direct access via index (not in LL)

Space Complexity → Memory used by algorithm

O(n) → Linear time; grows proportionally with n

Queue → FIFO structure; implementable via LL

Polynomial LL → Each node stores (coefficient, exponent)

Floyd's Algorithm → Cycle detection in linked list

Fast Pointer → Moves 2 steps at a time (Floyd's)

Reverse LL → Change all next pointers to previous

Sentinel Node → Dummy head/tail node to simplify ops

 Node Creation

```
// Create node and set data
Node* n = (Node*) malloc(sizeof(Node));
n->data = value;
n->next = NULL;
```

 Standard Traversal Loop

```
Node* temp = head;
while (temp != NULL) {
    printf("%d ", temp->data);
    temp = temp->next;
}
```

 Circular LL Traversal

```
Node* temp = head;
do {
    printf("%d ", temp->data);
    temp = temp->next;
} while (temp != head);
```

 Floyd's Cycle Detection

```
int hasCycle(Node* head) {
    Node* slow = head, *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return 1; // cycle!
    }
    return 0; // no cycle
}
```

 Merge Two Sorted Lists

```
Node* merge(Node* l1, Node* l2) {
    if (!l1) return l2;
    if (!l2) return l1;
    if (l1->data <= l2->data) {
        l1->next = merge(l1->next, l2);
        return l1;
    } else {
        l2->next = merge(l1, l2->next);
        return l2;
    }
}
```

 Reverse a Linked List

```
Node* prev = NULL;
Node* curr = head;
Node* next = NULL;
while (curr != NULL) {
    next = curr->next; // save next
    curr->next = prev; // reverse link
    prev = curr;      // move prev
    curr = next;      // move curr
}
head = prev;         // new head
```

 Find Middle (Slow-Fast)

```
Node* slow = head, *fast = head;
while (fast != NULL && fast->next != NULL) {
    slow = slow->next;
    fast = fast->next->next;
}
// slow now points to middle node
```

 HOW FLOYD'S WORKS

- **Slow pointer** moves 1 step at a time
- **Fast pointer** moves 2 steps at a time
- If cycle exists → they will *meet*
- If no cycle → fast reaches NULL
- Time: **O(n)** Space: **O(1)**

}

}

PATTERN 1: REVERSE LINKED LIST**Approach:** Use 3 pointers (prev, curr, next)**Time:** $O(n)$ **Space:** $O(1)$

- 1 Initialize prev=NULL, curr=head
- 2 Store next = curr→next
- 3 Reverse: curr→next = prev
- 4 Move: prev = curr, curr = next
- 5 Repeat until curr = NULL

PATTERN 2: FIND MIDDLE NODE**Approach:** Slow-fast pointer technique**Time:** $O(n)$ **Space:** $O(1)$

- 1 Start both pointers at head
- 2 Slow moves 1 step, fast moves 2 steps
- 3 When fast reaches end, slow = middle

```

[1]→[2]→[3]→[4]→[5]
slow↑          fast↑
(when fast=5, slow=3 = middle)
  
```

PATTERN 3: DETECT LOOP (FLOYD'S)**Approach:** Slow + fast pointer race**Time:** $O(n)$ **Space:** $O(1)$

- 1 Initialize slow=head, fast=head
- 2 Move slow by 1, fast by 2
- 3 If they meet → cycle exists
- 4 If fast reaches NULL → no cycle

```

[1]→[2]→[3]→[4]
                ↑-----↓
slow & fast meet at [3] or [4] → cycle!
  
```

PATTERN 4: MERGE TWO SORTED LISTS**Approach:** Compare heads, link smaller**Time:** $O(m+n)$ **Space:** $O(1)$

- 1 Compare heads of both lists
- 2 Attach smaller node to result
- 3 Move that list's pointer forward
- 4 Repeat until one list exhausted
- 5 Attach remaining list to result

Pattern Summary Table

Problem	Technique	Time	Space
Reverse LL	3-pointer (prev, curr, next)	$O(n)$	$O(1)$
Find Middle	Slow-Fast pointers	$O(n)$	$O(1)$
Detect Cycle	Floyd's algorithm	$O(n)$	$O(1)$
Merge Sorted Lists	Two-pointer comparison	$O(m+n)$	$O(1)$
Remove Nth from End	Two pointers, n-gap	$O(n)$	$O(1)$
Palindrome Check	Find mid + reverse + compare	$O(n)$	$O(1)$
Intersection of 2 LLs	Length difference trick	$O(m+n)$	$O(1)$

Q: What is a Linked List?

A: Collection of nodes connected via pointers; non-contiguous memory.

Q: What is the head pointer?

A: Pointer to the first node; entry point of the list.

Q: Time for inserting at beginning?

A: $O(1)$ – only head pointer changes.

Q: How does circular LL differ?

A: Last node's next points to head, not NULL.

Q: Advantage of DLL over SLL?

A: Bidirectional traversal; easier deletion without prev tracking.

Q: What is Floyd's algorithm used for?

A: Detecting cycles/loops in a linked list.

Q: How to find middle of LL?

A: Slow-fast pointer; when fast ends, slow is at middle.

Q: Traversal loop termination condition?

A: $\text{temp} \neq \text{NULL}$ (SLL) or $\text{temp} \neq \text{head}$ (CLL).

Q: Space complexity of linked list?

A: $O(n)$ – one node per element.

Q: Application of doubly LL?

A: Browser back/forward history, undo/redo in editors.

Q: How to reverse a singly LL?

A: Use 3 pointers: prev, curr, next; iteratively swap links.

Q: What is predecessor node?

A: The node immediately before the current node.

Q: Difference: array access vs LL access?**Q: What does each node contain?**

A: Data field + pointer(s) to next (and prev in DLL).

Q: What does head == NULL mean?

A: The linked list is empty.

Q: Time for inserting at end?

A: $O(n)$ – must traverse to last node.

Q: What is doubly linked list?

A: Each node has prev + next pointers; bidirectional traversal.

Q: Disadvantage of DLL?

A: Extra memory (2 pointers per node) and more complex code.

Q: How does Floyd's cycle detection work?

A: Slow ptr moves 1 step, fast ptr moves 2; they meet if cycle exists.

Q: What is traversal time in LL?

A: $O(n)$ – must visit all nodes sequentially.

Q: Why can't we do binary search on LL?

A: No random access; can't jump to mid without traversal.

Q: Application of circular LL?

A: Round-robin CPU scheduling, music playlist loop.

Q: What is a sentinel/dummy node?

A: Fake head/tail node to simplify edge cases in operations.

Q: Steps to insert at position p?

A: Traverse to $p-1$, then link newNode between $p-1$ and p .

Q: What is $O(1)$ vs $O(n)$?

A: $O(1)$ = constant time; $O(n)$ = time grows linearly with n .

Q: What does malloc() return?

A: Array = $O(1)$ random; LL = $O(n)$ sequential only.

A: Pointer to newly allocated heap memory block.

Q: What is free() used for?

A: Deallocates previously malloc'd memory to prevent leaks.

Q: Polynomial representation in LL?

A: Each node stores (coefficient, exponent, next pointer).

Q: How to detect end of singly CLL?

A: Stop when pointer returns to head (temp == head).

Q: Which LL is best for implementing Stack?

A: Singly LL – push/pop at head = $O(1)$.

Q: Why LL is better than array for insertion?

A: No shifting needed; only pointer update = $O(1)$ at head.

Q: Why is LL worse than array for access?

A: No index; must traverse from head = $O(n)$.

Q: Delete node given only its pointer in DLL?

A: $O(1)$ – adjust prev and next neighbors directly.

Q: What is adjacency list representation?

A: Each graph vertex has a LL of its neighboring vertices.

Q: What happens if you lose the head pointer?

A: Entire list becomes inaccessible → memory leak!

❌ COMMON MISTAKES TO AVOID

- **Losing head pointer** – always save head before modifying
- **Wrong NULL checks** – always check `if(head==NULL)` first
- **Forgetting to free memory** – causes memory leaks
- **Infinite loop in CLL traversal** – use `do-while` with `temp!=head`
- **Wrong pointer order during insertion** – always `newNode->next = old` BEFORE `head = newNode`
- **Not updating prev in DLL** – both prev and next must be updated
- **Using `while(temp)` in CLL** – temp never becomes NULL in circular!
- **Segfault on NULL dereference** – check before accessing `temp->next`

✅ WRITING TIPS FOR EXAM

- **Always draw diagrams** – visualize pointer changes step by step
- **Show before & after state** – makes your answer clear
- **Write step-wise algorithm** – numbered steps score better
- **State time complexity** – mention $O(n)$ or $O(1)$ for every operation
- **Handle edge cases** – empty list, single node, first/last node
- **Compare with arrays** – when asked "why LL", use the comparison table
- **Label pointer variables** – use prev, curr, temp, next clearly
- **Mention applications** – Stack, Queue, Round-robin, Browser history

🎯 High-Value Exam Topics (Most Likely Asked)

Topic	Marks Weight	Key Points
Insert/Delete Operations (all cases)	★★★★★	Draw diagram + write algo + state complexity
Arrays vs Linked Lists comparison	★★★★★	Memorize the comparison table
Singly vs Doubly vs Circular	★★★★☆	Pointer count, traversal, advantages
Floyd's Cycle Detection	★★★★☆	Slow/fast pointer concept + pseudocode
Reverse a Linked List	★★★★☆	3-pointer technique with diagram
Applications of Linked List	★★★★☆	Stack, Queue, Polynomial, Adjacency list
Node structure & malloc	★★★★☆	C struct definition + malloc syntax

📄 5-MINUTE PRE-EXAM CHECKLIST

- ☒ Know $O(1)$ vs $O(n)$ for each operation |
 ☒ Remember: INSERT at BEGIN = $O(1)$ |
 ☒ CLL uses do-while, not while |
 ☒ DLL: update BOTH prev and next |
 ☒ Floyd's: slow=1 step, fast=2 steps |
 ☒ Never lose the head pointer

🔗 Singly LL Operations

- Insert Begin → $O(1)$ · `newNode→next=head;`
`head=newNode`
- Insert End → $O(n)$ · traverse to last, `last→next=newNode`
- Insert Pos → $O(n)$ · traverse to p-1, adjust links
- Delete Begin → $O(1)$ · `head=head→next;` free old
- Delete End → $O(n)$ · traverse, `prev→next=NULL`
- Delete Node → $O(n)$ · `prev→next=curr→next;` free curr
- Traversal → $O(n)$ · `while(temp!=NULL) temp=temp→next`
- Search → $O(n)$ · linear scan from head

🔗 Doubly LL Key Points

- 2 pointers: `prev` + `next`
- Insert Begin → $O(1)$
- Delete by ptr → $O(1)$ (unique advantage!)
- Bidirectional traversal possible
- More memory: 2 extra pointers per node
- `Head→prev = NULL` always
- `Tail→next = NULL` always

🔗 Circular LL Key Points

- `last→next = head` (not NULL!)
- Traversal: `do-while (temp ≠ head)`
- No NULL pointer in the list
- Used for: Round-Robin, Buffers
- Insert: create node, link to head
- Single node: `node→next = itself`

🕒 Complexity Quick-Ref

- Access → $O(n)$
- Search → $O(n)$
- Insert Begin → $O(1)$ ✓
- Insert End/Mid → $O(n)$
- Delete Begin → $O(1)$ ✓
- Delete End/Mid → $O(n)$
- Reverse → $O(n)$ time, $O(1)$ space
- Floyd's Detect → $O(n)$ time, $O(1)$ space

⚖️ Arrays vs LL (Key Diffs)

- Array: contiguous · LL: non-contiguous
- Array: fixed size · LL: dynamic
- Array access: $O(1)$ · LL: $O(n)$
- Array insert: $O(n)$ · LL begin: $O(1)$
- Array: cache-friendly · LL: not
- Array: no extra ptr · LL: ptr overhead

🌱 Key Algorithms Summary

- **Reverse:** `prev=NULL`, swap links, prev becomes head
- **Middle:** `slow(+1) + fast(+2) → slow = mid`
- **Cycle:** `slow(+1) + fast(+2) → meet = cycle`
- **Merge:** compare heads, link smaller, recurse
- **Nth from end:** two ptrs with n-gap

📌 Critical Formulas / Conditions

- Empty list: `head = NULL`
- Single node: `head→next = NULL`
- Last node (SLL): `temp→next = NULL`
- End of CLL: `temp = head`
- New node at begin: `newNode→next = head; head = newNode`
- Space per node (SLL): `data + 1 ptr`
- Space per node (DLL): `data + 2 ptrs`

🌐 Applications – One-Liners

- Stack → SLL (push/pop at head)
- Queue → SLL (enqueue tail, dequeue head)
- Browser history → DLL (back/forward)
- Round-Robin → Circular LL
- Polynomial → SLL (coeff + exp nodes)
- Adjacency List → Array of LLs (graphs)
- Undo/Redo → DLL (editor)

GOLDEN RULES FOR EXAM SUCCESS

Always draw diagrams

Show before & after state

Never lose HEAD

Save it before any changes

State complexity

Mention $O(1)$ or $O(n)$ always

Step-wise algorithms

Number every step

Handle edge cases

Empty list + single node

Use comparison tables

Arrays vs LL in 2 mins